

# D2 — Enterprise Audit & Orchestration Report

ModSwap • Flutter (Android + Web) • Firebase • 31 May 2026

*Prepared by the ModSwap Engineering Team • Private — not for redistribution*

This report covers four mandated audit areas: multi-agent orchestration workflow, domain model design and state management rationale, role-based access control matrix and Firestore security rules, and the observability infrastructure and rollback strategy for the live semantic search feature flag.

---

**ModSwap Engineering Team**

Sorasit Laiget • Veerachai Sribuatim • Poowrin  
Trijakpranee  
Chakkrapat Supanith • Siwakon Wanichsujit

KMUTT • Faculty of Information  
Technology  
Academic Year 2025–2026

# Contents

|          |                                                       |          |
|----------|-------------------------------------------------------|----------|
| <b>1</b> | <b>Agent Workflow &amp; Multi-Agent Orchestration</b> | <b>2</b> |
| 1.1      | The Four-Agent Model                                  |          |
| 1.2      | Dispatch Protocol                                     |          |
| 1.3      | Context Drift Challenges and Mitigations              |          |
| 1.4      | Handoff Management                                    |          |
| <b>2</b> | <b>Architecture &amp; Data</b>                        | <b>3</b> |
| 2.1      | Architecture                                          |          |
| 2.2      | Domain Models                                         |          |
| 2.3      | Firestore Collection Hierarchy                        |          |
| 2.4      | State Management — Riverpod                           |          |
| <b>3</b> | <b>Security Matrix</b>                                | <b>4</b> |
| 3.1      | RBAC Matrix                                           |          |
| 3.2      | Firestore Security Rules                              |          |
| 3.3      | Firebase Storage Rules                                |          |
| 3.4      | Biometric Authentication                              |          |
| 3.5      | Known Security Debts                                  |          |
| <b>4</b> | <b>Observability &amp; Rollback</b>                   | <b>6</b> |
| 4.1      | Structured Logging (AppLogger)                        |          |
| 4.2      | Crashlytics Implementation                            |          |
| 4.3      | Feature Flag System                                   |          |
| 4.4      | Rollback Plan — enable_semantic_search                |          |

# 1. Agent Workflow & Multi-Agent Orchestration

## The Four-Agent Model

Development of ModSwap is assisted by a four-agent orchestration system defined in `.claude/agents/` and governed by `CLAUDE.md`. Each agent is scoped to a single concern and may not edit another agent's domain.

| Agent             | Concern                                                         | Tools                  | Edit Scope                                                                            |
|-------------------|-----------------------------------------------------------------|------------------------|---------------------------------------------------------------------------------------|
| architect         | Layering, data models, API contracts, trade-off evaluation      | Read, Glob, Grep, Bash | Diagnose only                                                                         |
| flutter_engineer  | Screens, widgets, providers, navigation, services               | +Edit, Write           | lib/screen/ ,<br>lib/widgets/ ,<br>lib/services/ ,<br>lib/models/ ,<br>lib/providers/ |
| qa_engineer       | Unit tests, widget tests, flutter analyze, accessibility sweeps | +Edit, Write           | test/                                                                                 |
| security_reviewer | Auth, Firestore rules, Storage rules, secrets, CVEs             | Read, Glob, Grep, Bash | Diagnose only                                                                         |

`architect` and `security_reviewer` are read-only; they diagnose and report but cannot modify source files. `flutter_engineer` and `qa_engineer` hold write access, each constrained to its own path namespace. Complex tasks must pass through Plan Mode ( `architect` ) before any implementation begins.

## Dispatch Protocol

All dispatches follow a topic-map that matches user intent against a keyword table:

| User intent                            | Agents dispatched             |
|----------------------------------------|-------------------------------|
| Add/fix a screen, widget, or route     | flutter_engineer              |
| Firestore model or API endpoint change | architect + flutter_engineer  |
| Auth flow or security-sensitive change | architect + security_reviewer |
| Security review, rules audit, CVE      | security_reviewer             |
| Tests or flutter analyze               | qa_engineer                   |
| Layering or architecture concern       | architect                     |
| New feature spanning data and UI       | architect + flutter_engineer  |
| Pre-merge / pre-release gate           | All four agents               |

Every prompt is self-contained and includes: (1) target file paths the agent must read first, (2) the original ask verbatim, (3) which `CLAUDE.md` section to read, and (4) the expected report shape. The orchestrator states the routing decision in one line before dispatching so the user can redirect.

## Context Drift Challenges and Mitigations

Because agents start cold on each dispatch, they risk drifting from established patterns or creeping outside their scope. Three structural mitigations address this:

- **Hard edit-scope boundaries.** Each agent's definition declares a *What you own* list. Violations are out-of-scope and flagged by the orchestrator.

- **Diagnose-only agents.** `architect` and `security_reviewer` hold no write tools for production code, so even drifted output cannot be committed — it can only produce a report for human review.
- **CLAUDE.md as single source of truth + flutter analyze gate.** Every agent reads `CLAUDE.md` first; its explicit DO-NOT-DO list serves as a checklist. A non-zero analyzer exit is an objective signal of drift. The `qa_engineer` must report a clean `flutter analyze` before claiming completion.

## Handoff Management

After all agents return, the orchestrator produces a single consolidated report grouped by severity (**Critical / High / Medium / Info**), with each finding tagged by the originating agent. The `architect` verdict prevails on structural disagreements; the `security_reviewer` verdict prevails on data-exposure disagreements. The agent that writes code must not be the agent that approves it — `flutter_engineer` code requires sign-off from `security_reviewer` or `qa_engineer` before merge.

## 2. Architecture & Data

### Architecture

ModSwap has completed its migration to **Clean Architecture**. The codebase is organized into three strict layers enforced by the `architect` agent:

| Layer        | Path                           | Constraint                                                                                                                  |
|--------------|--------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| Domain       | <code>lib/domain/</code>       | Zero imports of <code>flutter/</code> , <code>firebase_*</code> , <code>dio</code> , <code>dart:io</code> . Pure Dart only. |
| Data         | <code>lib/data/</code>         | Implements domain repository interfaces. Owns SDK adapters (Firebase, Dio).                                                 |
| Presentation | <code>lib/presentation/</code> | Riverpod notifiers, screens, widgets. Depends on domain use cases only.                                                     |

The domain layer purity constraint is verified by the `architect` agent on every audit: `grep -r "import 'package:flutter\|firebase\|dio\|dart:io" lib/domain/` must return no results.

The old `lib/providers/` and `lib/services/` directories have been replaced by the Clean Architecture layers. `lib/services/` is retained only for legacy compatibility during the remaining screen-level migration; new code writes exclusively to `lib/presentation/` and `lib/domain/`.

### Domain Models

| Model           | Key Fields                                                                                                                                                                                                                                                                                                                                                                                                          | Firestore Path                              |
|-----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------|
| User            | <code>uid</code> , <code>displayName</code> , <code>email</code> , <code>studentId</code> , <code>lineId</code> , <code>rating</code> , <code>totalReviews</code> , <code>totalTrades</code> , <code>avatarURL</code>                                                                                                                                                                                               | <code>users/{uid}</code>                    |
| Listing         | <code>id</code> , <code>ownerId</code> , <code>ownerName</code> , <code>ownerStudentId</code> , <code>ownerLineId</code> , <code>title</code> , <code>description</code> , <code>category</code> , <code>type</code> , <code>price</code> , <code>images</code> , <code>thumbnailURL</code> , <code>condition</code> , <code>meetingPoint</code> , <code>state</code> , <code>views</code> , <code>embedding</code> | <code>listings/{id}</code>                  |
| Deal            | <code>id</code> , <code>listingId</code> , <code>sellerId</code> , <code>buyerLineId</code> , <code>dealType</code> , <code>finalPrice</code> , <code>whatIGotReturn</code> , <code>swapItemPhotoURL</code> , <code>dateCompleted</code>                                                                                                                                                                            | <code>deals/{id}</code>                     |
| PendingRating   | <code>id</code> , <code>buyerUid</code> , <code>sellerId</code> , <code>sellerName</code> , <code>listingId</code> , <code>listingTitle</code>                                                                                                                                                                                                                                                                      | <code>pendingRatings/{id}</code>            |
| AppNotification | <code>id</code> , <code>type</code> , <code>title</code> , <code>body</code> , <code>isRead</code> , <code>createdAt</code> , <code>deepLinkTarget?</code> , <code>data?</code>                                                                                                                                                                                                                                     | <code>users/{uid}/notifications/{id}</code> |

`Listing.embedding` is a 768-dimensional float vector produced by Gemini `embedding-001`; it is written by a Cloud Functions `onCreate` trigger and is never set from the client. `Listing.state` follows the FSM `draft` → `published` → `sold`; transitions are enforced server-side in `ListingsService`.

## Firestore Collection Hierarchy

The schema uses a mix of top-level collections and owner-scoped subcollections:

```
users/{uid}
  /notifications/{notifId}    ← real-time notification inbox
  /devices/{deviceId}         ← registered device tokens
  /wishlist/{listingId}       ← saved listings (document ID = listing ID)

listings/{id}                 ← top-level; queryable by category, state, ownerId

deals/{id}                    ← top-level; each deal references a listingId

pendingRatings/{id}           ← top-level; buyer reads own, seller creates
```

Listing images, profile avatars, and deal swap photos are stored in Firebase Storage (not Firestore); only the download URL is persisted in Firestore.

Top-level placement for `listings`, `deals`, and `pendingRatings` avoids the collection-group query complexity that sub-collections introduce in Firestore Security Rules and enables simple ownership queries ( `where('ownerId', isEqualTo: uid)` ) without a `collectionGroup` index.

## State Management — Riverpod

`setState` is prohibited for shared state. The project uses `flutter_riverpod` with `riverpod_annotation` code generation throughout. The old `ChangeNotifier` providers have been fully removed.

| Provider                                       | Type                                    | Purpose                                                                           |
|------------------------------------------------|-----------------------------------------|-----------------------------------------------------------------------------------|
| <code>authNotifierProvider</code>              | <code>@Riverpod(keepAlive: true)</code> | Auth FSM state machine; owns <code>AuthStateData</code> + <code>AuthStatus</code> |
| <code>appRouterProvider</code>                 | <code>@Riverpod(keepAlive: true)</code> | <code>GoRouter</code> instance; watches auth state for redirect                   |
| <code>notificationNotifierProvider</code>      | <code>@Riverpod(keepAlive: true)</code> | Live notification stream; cancels <code>StreamSubscription</code> on uid change   |
| <code>themeNotifierProvider</code>             | <code>@Riverpod(keepAlive: true)</code> | Light/dark mode preference                                                        |
| DI providers (in <code>providers.dart</code> ) | <code>Provider&lt;T&gt;</code>          | Data sources, repository impls, use case instances                                |

`keepAlive: true` on `authNotifierProvider` and `appRouterProvider` prevents `GoRouter` from being reconstructed on unrelated widget rebuilds. `RouterRefreshNotifier` (a thin `ChangeNotifier` adapter) bridges the Riverpod auth stream to `GoRouter`'s `refreshListenable` — the only remaining `ChangeNotifier` in the codebase.

Navigation uses **GoRouter** declaratively. The router performs auth-state-driven redirects across the full `AuthStatus` FSM: `initializing` → `unauthenticated` → `emailUnverified` → `profileIncomplete` → `authenticated`. Path-based routes cover all screens ( `/login`, `/register`, `/verify-email`, `/complete-profile`, `/home`, `/listing/{id}`, `/post-item`, `/edit-profile`, `/change-password` ). Six screens still use `Navigator.push` for internal modal flows; these are tracked for `GoRouter` migration in the next sprint.

### 3. Security Matrix

#### RBAC Matrix

| Role                  | users                                | listings        | deals                | pendingRatings                             | notifications   | Storage                                          |
|-----------------------|--------------------------------------|-----------------|----------------------|--------------------------------------------|-----------------|--------------------------------------------------|
| Unauthenticated       | Deny                                 | Deny            | Deny                 | Deny                                       | Deny            | Deny                                             |
| KMUTT Student (self)  | Read + update name/linelid/avatarURL | Full CRUD (own) | Create own; read own | Create (as seller); read/delete (as buyer) | Full CRUD (own) | Write own prefix; public read (listings/avatars) |
| KMUTT Student (other) | Read (profile lookup for linelid)    | Read published  | Deny                 | Deny                                       | Deny            | Public read only                                 |
| Cloud Function        | Admin SDK (bypasses rules)           | Admin SDK       | Admin SDK            | Admin SDK                                  | Admin SDK       | Admin SDK                                        |

The `rating`, `totalReviews`, and `updatedAt` fields on `users/{uid}` are the only fields any authenticated user may update on another user's document — enforced by `diff().affectedKeys().hasOnly(['...'])` in Firestore rules.

#### Firestore Security Rules

Source: `firestore.rules`

Three helper functions encapsulate repeated logic: `isAuthenticated()` (checks `request.auth != null`), `isKmuttEmail()` (verifies `@mail.kmutt.ac.th` domain on the Firebase ID token email claim), and `isOwner(userId)` (compares `request.auth.uid == userId`).

**`users/{uid}`** : Authenticated users may read any profile (needed for seller lookup and linelid display). Update is split: the owner may update profile fields; any authenticated user may update only `rating`, `totalReviews`, and `updatedAt` on another user's document via `diff().affectedKeys().hasOnly(['rating', 'totalReviews', 'updatedAt'])`. Create and delete are denied from the client — user documents are managed exclusively by the Cloud Functions `on-user-create` trigger.

**`listings/{id}`** : Create enforces `isKmuttEmail()` and binds `ownerId` to `request.auth.uid`. Published listings are readable by all authenticated users; draft listings are owner-only. Delete is owner-only and only from `draft` state.

**`pendingRatings/{id}`** : The seller creates the pending rating record after marking a deal complete. The buyer reads and deletes their own record (keyed by `buyerUid`). Update is restricted to the seller (`sellerId == request.auth.uid`) for idempotent upserts using `listingId` as the document ID.

**`users/{uid}/notifications/{notifId}`** : Owner reads, creates, and deletes. Any KMUTT-verified user may create a notification (allows buyer→seller notification path). Update is restricted to `isRead` field only via `diff().affectedKeys().hasOnly(['isRead'])`.

**Default deny:** A catch-all `match /{document=**}` denies all reads and writes for any unmatched path.

#### Firebase Storage Rules

Source: `storage.rules`

| Path                                                  | Read   | Write                                                    |
|-------------------------------------------------------|--------|----------------------------------------------------------|
| <code>listings/{userId}/{listingId}/{fileName}</code> | Public | KMUTT student; owner only; ≤ 10 MB; <code>image/*</code> |
| <code>avatars/{userId}/{fileName}</code>              | Public | KMUTT student; owner only; ≤ 5 MB; <code>image/*</code>  |

| Path                                               | Read          | Write                                                    |
|----------------------------------------------------|---------------|----------------------------------------------------------|
| <code>deals/{userId}/{listingId}/{fileName}</code> | Authenticated | KMUTT student; owner only; ≤ 10 MB; <code>image/*</code> |
| <code>/{"allPaths==**"}</code>                     | Deny          | Deny                                                     |

Listing and avatar images are intentionally public-read (required for unauthenticated preview and CDN caching). Deal swap photos are restricted to authenticated users because they may contain personal items and meetup context.

## Biometric Authentication

Source: `lib/services/biometric_service.dart`

Biometric login is an optional acceleration layer — never a sole credential. The implementation uses `local_auth` for authentication and `flutter_secure_storage` for credential storage:

- **Android:** `AndroidOptions(encryptedSharedPreferences: true)` — credentials stored in encrypted `SharedPreferences` backed by Android Keystore.
- **iOS:** `IOSOptions(accessibility: KeychainAccessibility.first_unlock)` — credentials stored in Keychain, accessible after first device unlock.
- `biometricOnly: false` in `AuthenticationOptions` allows device PIN/passcode as fallback when biometric hardware is unavailable or the attempt fails.
- `stickyAuth: true` prevents the system from cancelling authentication if the app is briefly backgrounded.

The raw password is stored in secure storage to support the Firebase `signInWithEmailAndPassword` call on biometric success. The stored credentials are cleared on `clearCredentials()` (logout or biometric disable).

## Known Security Debts

Documented in `CLAUDE.md` as deliberately deferred for the current development phase:

1. **`lineId` visible to all authenticated users.** The current Firestore rule allows any authenticated user to read any `users/{uid}` document, including `lineId`. This is necessary for the buyer→seller contact flow but exposes contact details beyond what is strictly needed.
2. **`deepLinkTarget` in notifications is not validated.** A crafted notification document could supply an arbitrary deep-link path. Client-side route validation before navigation is not yet implemented.
3. **No rate limiting on Cloud Functions endpoints.** The Functions are protected by Firebase ID token verification but have no per-user request throttle, making them a DoS target.
4. **`studentId` stored in plaintext.** Student IDs are stored as plain strings in Firestore. They are protected by Firestore rules but are not encrypted at the field level.



## 4. Observability & Rollback

### Structured Logging (AppLogger)

Source: `lib/utils/logger.dart`

All logging is centralised through the static `AppLogger` class (wraps `dart:developer`):

```
AppLogger.d(message, tag: 'AUTH');    // Level 500 – suppressed in release
AppLogger.i(message, tag: 'LISTING');  // Level 800 – always emitted
AppLogger.w(message, tag: 'SEARCH');   // Level 900 – always emitted
AppLogger.e(message, tag: 'SEARCH',    // Level 1000 – mirrors to Crashlytics
  error: e, stackTrace: st);
```

`AppLogger.e()` with a non-null `error` object automatically calls `FirebaseCrashlytics.instance.recordError()` in release mode, surfacing handled non-fatal errors alongside fatal crashes in the Crashlytics dashboard. Debug builds use `dart:developer`'s native `log()` which routes to the IDE console with tag filtering. Tags are arbitrary strings (e.g. `AUTH`, `LISTING`, `SEARCH`, `DEALS`, `RATING`) used for log filtering in Crashlytics and the device console.

### Crashlytics Implementation

Sources: `lib/main.dart`, `lib/utils/logger.dart`

All Crashlytics instrumentation sites are wrapped in `if (!kIsWeb)`; collection is further gated by `setCrashlyticsCollectionEnabled(!kDebugMode)` so crashes are never reported during local development.

| # | Source                         | API call                                        | What it captures                                                                             |
|---|--------------------------------|-------------------------------------------------|----------------------------------------------------------------------------------------------|
| 1 | <code>main.dart</code>         | <code>recordFlutterFatalError</code>            | Flutter widget build and render errors ( <code>FlutterError.onError</code> )                 |
| 2 | <code>main.dart</code>         | <code>recordError(fatal: true)</code>           | Platform-thread and Dart isolate errors ( <code>PlatformDispatcher.instance.onError</code> ) |
| 3 | <code>main.dart</code>         | <code>recordError(fatal: true)</code>           | Fallback — errors not caught by sites 1 or 2 ( <code>runZonedGuarded</code> )                |
| 4 | <code>utils/logger.dart</code> | <code>recordError(fatal: false) + reason</code> | Non-fatal handled errors via <code>AppLogger.e()</code>                                      |

Site 4 means every `AppLogger.e()` call with a non-null `error` object is automatically mirrored to Crashlytics with the tag and message as the `reason` field, surfacing non-fatal handled errors alongside fatal crashes without explicit Crashlytics calls at each call site.

### Feature Flag System

Source: `lib/services/remote_config_service.dart`

`RemoteConfigService` wraps `FirebaseRemoteConfig` with typed getters. It is constructed as a singleton and initialized once at app startup. Initialization is failure-safe: a `try/catch` logs the error via `AppLogger.e()` and falls through to hardcoded defaults so the app never blocks on Remote Config failure ( `fetchTimeout` : 10 s; `minimumFetchInterval` : 60 min in production, 0 s in debug).

| Remote Config key                   | Default           | Effect                                                                                                                                    |
|-------------------------------------|-------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| <code>enable_semantic_search</code> | <code>true</code> | Master switch for the Gemini AI semantic search pipeline. <code>false</code> = keyword-only text search; no embedding API calls are made. |

The flag is checked in `HomeScreen` before each search invocation. When `false`, the app falls back to the existing keyword-based Firestore query ( `where('title', isGreaterThanOrEqualTo: query)` ).

### Rollback Plan — `enable_semantic_search`

The Gemini semantic search pipeline involves an outbound HTTPS call to the Gemini Embedding API, a Cloud Functions invocation for embedding generation, and a Firestore cosine-similarity query over a large vector index. Any of these can spike in latency or fail under quota pressure.

The flag provides a zero-redeploy kill switch:

1. **Trigger.** Crashlytics `SEARCH` -tagged error spike, P95 home-feed latency regression, or Gemini API quota exhaustion reported in Google Cloud Console.
2. **Action.** Firebase Console → Remote Config → `enable_semantic_search` → `false` → Publish Changes.
3. **Propagation.** Clients running in production pick up the change within 60 minutes (the `minimumFetchInterval` ). New cold starts fetch immediately (within 10 s).
4. **Effect.** `HomeScreen` routes all search queries to the keyword path. The Gemini Embedding API and the semantic Cloud Function are not called. Home feed loads from Firestore only — no external API dependency.
5. **Recovery.** After the root-cause fix is deployed (quota raised, embedding index rebuilt, or latency regression patched), flip `enable_semantic_search` back to `true` . Clients update within 60 minutes; no client redeploy is required.

More broadly, `CLAUDE.md` classifies all engineering actions by reversibility: **R0** (irreversible — stop and confirm first), **R1** (costly to reverse — proceed with documented justification), and **R2** (easily reversed — just do it). Remote Config flag changes are R2. Firestore index changes and Storage rule changes are R1. Firestore document deletes and auth config changes are R0.